

Software Instruction Manual

Chapter 7: Dynamic Modeling of the Catalytic Reactor for Sulfur Dioxide Oxidation

This chapter provides a comprehensive description of the dynamic model developed for the catalytic reactor used in the oxidation of sulfur dioxide (SO_2) to sulfur trioxide (SO_3) in the sulfuric acid manufacturing process. The model is derived from steady-state equations outlined in standard chemical engineering references, such as Fogler (Section R8.4) [1], and extends them to include time-dependent terms for capturing transient behavior. It is intended for use in process simulation, control system design, and operational troubleshooting within the plant.

The model assumes plug-flow conditions with variable superficial velocity u , ideal gas behavior, and a pseudo-homogeneous approach. Given the single packed bed configuration with a large diameter (in feet), radial temperature gradients and wall heat transfer are considered negligible, resulting in an adiabatic assumption. Although the actual process includes inter-stage cooling (as depicted in Figures R8.4-1 and R8.4-2 from reference documentation [1]), this simplification emphasizes reaction-generated heat effects, which predominate in large-diameter beds where the surface-to-volume ratio is low and boundary heat losses are minimal.

The reaction rate r is expressed per unit mass of catalyst, as defined in Equation (R8.4-1) [1]. The bed porosity ϵ (void fraction) is approximately 0.45 for the described Reymersholm V/205 catalyst on pumice carrier (cylindrical pellets: 8 mm diameter and length, bulk density $\rho_b = 33.8 \text{ lb/ft}^3$). Units employ "lbmol" without spaces (e.g., lbmol/ft³ for concentrations).

The following sections present the partial differential equations (PDEs), key definitions, the necessity of hyperbolic PDEs for non-steady-state plug-flow reactors (PFRs), boundary and initial conditions (including discretization indices), spatial discretization procedures, numerical solution methods, and implementation guidance. For operational context, refer to the typical sulfuric acid plant flowsheet (Figure R8.4-1 [1]), which includes a feed of 7900 lbmol/h (11% SO_2 , 10% O_2 , 79% inerts) to produce 1000-2400 tons/day of acid, with volumetric flow rates of 75-100 ft³/s (at NTP) over the catalyst.

7.1 Mass Balance PDEs for Species Concentrations

For each species i (SO_2 , O_2 , SO_3 , N_2):

$$\epsilon \frac{\partial C_i}{\partial t} + \frac{\partial(uC_i)}{\partial z} = \nu_i \rho_b r$$

Explanation: This equation accounts for accumulation ($\partial C_i / \partial t$), convective transport ($\partial(uC_i) / \partial z$), and generation/consumption via reaction ($\nu_i \rho_b r$).

Variables:

- C_i : Concentration of species i (lbmol/ft³)
- u : Superficial gas velocity (ft/s), computed as: $u = u_0 \frac{(1 + \varepsilon_g X) P_0 T}{P T_0}$ where $\varepsilon_g \approx -0.055$ (gas expansion factor from the document), and u_0 is the inlet velocity
- ν_i : Stoichiometric coefficient ($\nu_{\text{SO}_2} = -1$, $\nu_{\text{O}_2} = -1/2$, $\nu_{\text{SO}_3} = 1$, $\nu_{\text{N}_2} = 0$)
- r : Reaction rate per unit catalyst mass (lbmol/(lb_{cat}·s·atm^{1/2})), given by: $r = k \frac{P_{\text{O}_2}}{P_{\text{SO}_2} \left(1 + K_p \frac{P_{\text{SO}_3}}{P_{\text{O}_2}^{1/2}}\right)^2}$ with partial pressures $P_i = C_i R_g T$ (in atm)
- k and K_p : Rate and equilibrium constants from Equations (ER8.4-2) and (ER8.4-1) [1], respectively

****Note:**** Local conversion X can be derived from concentrations as:

$$X = 1 - \frac{C_{\text{SO}_2}}{C_{\text{SO}_2,0}} \cdot (1 + \varepsilon_g X)$$

but solving in terms of concentrations avoids implicit dependencies.

7.2 Energy Balance PDE (Simplified by Removing Cooling)

Assuming thermal equilibrium between fluid and solid phases, and adiabatic conditions (no cooling term):

$$[\varepsilon \rho_g C_{p,g} + (1 - \varepsilon) \rho_s C_{p,s}] \frac{\partial T}{\partial t} + u \rho_g C_{p,g} \frac{\partial T}{\partial z} = (-\Delta H_{\text{Rx}}) \rho_b r$$

Explanation: This captures thermal accumulation ($\partial T / \partial t$), convective heat transport ($u \partial T / \partial z$), and heat release from reaction. The cooling term has been removed for simplification, as it becomes negligible in large-diameter packed beds where heat transfer to the walls is minimal compared to axial convection and reaction heat.

Variables:

- T : Temperature (R)
- ρ_g : Gas density (lb/ft³), approximated as $\rho_g = \frac{P M_{\text{avg}}}{R_g T}$ where M_{avg} is the average molecular weight (mixture-dependent)
- $C_{p,g}$: Mixture-averaged gas heat capacity (Btu/lb·°R), using expressions for individual $C_{p,i}$ from the document [1]

- $\rho_s, C_{p,s}$: Solid particle density and heat capacity (Btu/lb·°R); $\rho_s = \rho_b/(1 - \varepsilon)$, with $C_{p,s} \approx 0.2$ Btu/lb·°R for vanadia catalysts (approximate if not specified; may neglect if fluid heat capacity dominates)
 - $\Delta H_{R,x}$: Heat of reaction (Btu/lbmol SO₂), from Equation (ER8.4-8) [1]
-

7.3 Momentum Balance (Quasi-Steady State)

Since pressure adjusts rapidly compared to the time scales of concentration and temperature transients, the momentum equation is treated as quasi-steady. The pressure drop along the reactor is modeled using the Ergun equation, adapted for variable gas density and temperature:

$$\frac{dP}{dz} = -\frac{G}{g_c \rho_b D_p} \left[\frac{150(1 - \varepsilon)\mu}{\varepsilon^3 D_p} + 1.75(1 + \varepsilon_g X) \frac{G}{\varepsilon^3} \right] \cdot \frac{T}{T_0} \cdot \frac{P_0}{P}$$

Explanation: This equation accounts for frictional pressure losses in the packed bed, incorporating the effects of changing gas density, temperature, and mole number (via the expansion factor $\varepsilon_g X$). The first term in the bracket represents viscous losses (Blake-Kozeny), while the second represents inertial losses (Burke-Plummer).

Variables:

- P : Pressure (atm)
- G : Mass velocity (lb/ft²·h), from the document as $G = \sum F_{i,0} M_i / A_c \approx 1307.6$ lb/ft²·h (M_i : molecular weight of species i) [1]
- g_c : Gravitational constant (32.174 lb·ft/lb_f·s²)
- D_p : Effective particle diameter (0.015 ft, from document [1])
- μ : Gas viscosity (approximately 0.090 lb/(ft·h) at 1400 R, weakly temperature-dependent)
- ρ_g : Gas density (lb/ft³), $\rho_g = PM_{\text{avg}}/(R_g T)$
- A_c : Cross-sectional area of the bed (ft²), scaled for the large-diameter single bed (computed from total volumetric flow and inlet velocity) [1]

Conversion to Catalyst Weight Coordinate

The pressure derivative is frequently expressed with respect to catalyst weight W rather than axial position z , since $W = \rho_b A_c z$ (with ρ_b the bulk density and A_c the cross-sectional area) [1]. The transformation is:

$$\frac{dP}{dW} = \frac{dP}{dz} \cdot \frac{dz}{dW} = \frac{1}{\rho_b A_c} \cdot \frac{dP}{dz}$$

Substituting the Ergun expression for dP/dz yields the weight-based form:

$$\frac{dP}{dW} = -\frac{G}{g_c A_c D_p} \left[\frac{150(1 - \varepsilon)\mu}{\varepsilon^3 D_p} + 1.75(1 + \varepsilon_g X) \frac{G}{\varepsilon^3} \right] \cdot \frac{T}{T_0} \cdot \frac{P_0}{P}$$

This form is convenient for integration along catalyst weight in both steady-state and dynamic simulations, as W directly correlates with the amount of catalyst exposed to the reacting stream [1].

7.4 Necessity of Hyperbolic PDEs for Non-Steady-State Plug-Flow Reactors

In a non-steady-state plug-flow reactor (PFR), the system exhibits dynamic behavior where variables such as concentration, temperature, and pressure vary both spatially (along the reactor length z) and temporally (with time t). This necessitates the use of hyperbolic PDEs, which arise naturally from the conservation laws under the plug-flow assumption.

Hyperbolic PDEs are characterized by wave-like propagation of information, reflecting how disturbances (e.g., changes in inlet flow, composition, or temperature) travel downstream at finite speeds determined by the gas velocity u . Unlike parabolic PDEs (which include diffusion and imply infinite propagation speed), hyperbolic equations ensure causality—effects cannot precede causes—and accurately capture sharp fronts or transients in fast-reacting systems like the SO_2 oxidation, where exothermic reactions can lead to rapid temperature rises or hotspots.

For steady-state PFRs, ordinary differential equations (ODEs) suffice, as time derivatives vanish; however, in dynamic scenarios, such as startup, shutdown, or feed perturbations in a sulfuric acid plant, hyperbolic PDEs are essential to model the evolution of profiles without unphysical smoothing or instability, ensuring reliable predictions for control and safety [1].

7.5 Boundary and Initial Conditions

Boundary Conditions (at $z = 0$):

$$C_i(0, t) = C_{i,0}(t), \quad T(0, t) = T_0(t), \quad P(0, t) = P_0(t)$$

(inlet values from the document or upstream plant model [1]; these may be time-dependent during dynamic operation).

Initial Conditions (for each integration interval $[t_k, t_{k+1}]$):

The state profiles at time t_k across all slices:

$$C_i(z_j, t_k), \quad T(z_j, t_k), \quad P(z_j, t_k) \quad \text{for } j = 1, \dots, N$$

- **For the first interval ($t_0 = 0$)** : Use the steady-state profiles $C_i(z, 0)$, $T(z, 0)$, $P(z, 0)$ obtained by integrating the steady-state ODEs along z (set time derivatives to zero), as provided in the document (e.g., from Figures ER8.4-1.1 or Polymath output [1])
- **For subsequent intervals ($k > 1$)** : Use the converged state profiles computed at the end of the previous time step (i.e., at t_k)

Here, j denotes the spatial slice index ($j = 1, 2, \dots, N$), corresponding to the discretized positions along the reactor length ($z_j = j \cdot \Delta z$). k denotes the temporal index ($k = 0, 1, 2, \dots$) labeling discrete time points t_k at which major solution steps occur. This formulation ensures continuity of the solution across time and correctly represents the propagation of transients (such as temperature or conversion fronts) through the catalyst bed [1].

7.6 Spatially Discretizing the PDEs into ODEs

To facilitate numerical solution of the hyperbolic PDEs described in Sections 7.1-7.3, the method of lines (MOL) is employed for spatial discretization. This approach approximates the spatial derivatives in the PDEs, transforming the system into a set of ordinary differential equations (ODEs) in time, which can then be integrated using appropriate solvers.

The reactor length L is divided into N discrete slices along the axial direction z , with each slice having a uniform width $\Delta z = L/N$. The state variables (concentrations C_i , temperature T , and pressure P) are evaluated at discrete points $z_j = j \cdot \Delta z$ for $j = 1, 2, \dots, N$ (typically at the center or downstream face of each slice, depending on the numerical scheme).

Role of Residence Time in the Discretization

In a plug-flow reactor, the residence time τ is the time a fluid element spends traveling from the inlet to a given axial position z . For constant velocity, $\tau = z/u$, but in this model, velocity u varies along the reactor due to mole change (via $\varepsilon_g X$), temperature, and pressure effects. Thus, the local residence time to reach position z_j is:

$$\tau(z_j) = \int_0^{z_j} \frac{dz'}{u(z')}$$

This residence time concept is central to understanding the dynamic behavior of the discretized system. Each slice j has an effective local residence time $\Delta\tau_j \approx \Delta z/u_j$, which represents the average time a fluid element

spends in that slice. In the steady state, the extent of reaction in slice j depends directly on $\Delta\tau_j$ (longer residence time $\rightarrow$ more conversion). In the dynamic model, the accumulation term $\varepsilon\partial C_i/\partial t$ in each slice reflects how concentration changes over real time t due to reaction during the brief passage through $\Delta\tau_j$.

When a transient disturbance enters at $z = 0$ (e.g., a step change in inlet temperature or composition), it propagates downstream as a wave traveling at velocity u . The time required for this disturbance to reach slice j is approximately the cumulative residence time $\tau(z_j)$. This explains the hyperbolic (wave-like) nature of the system: information travels at finite speed, and the discretized ODEs naturally capture the delay in response at downstream slices [1].

Conceptualization as a Cascade of CSTRs

An alternative and highly intuitive way to view the discretized model is as a series (cascade) of continuous stirred-tank reactors (CSTRs), one for each spatial slice. Each CSTR corresponds to a well-mixed volume $\Delta V = A_c\Delta z$ (where A_c is the cross-sectional area), with the following key characteristics:

- The volume of each CSTR is fixed, corresponding to the physical slice volume
- Inlet to CSTR j is the outlet from CSTR $j - 1$, with flow rate uA_c (accounting for variable velocity)
- Each CSTR is assumed perfectly mixed, so concentration, temperature, and reaction rate are uniform within the slice (this is the key approximation of the finite-volume or finite-difference discretization)
- The accumulation term ($\varepsilon\partial C_i/\partial t$) in the mass balance becomes the rate of change of moles in the CSTR volume
- The convective term ($\partial(uC_i)/\partial z$) becomes the net molar flow into and out of the CSTR
- The reaction term ($\nu_i\rho_b r$) is evaluated at the uniform conditions inside the CSTR

Impact of Changing Density on Residence Time and Equations

The gas density $\rho_g = PM_{\text{avg}}/(R_gT)$ (lb/ft³) varies along the reactor due to changes in total lbmols (from reaction-induced mole reduction, $\varepsilon_g < 0$), temperature T , and pressure P . For this highly exothermic reaction, the heat release significantly increases T , which decreases ρ_g (since density is inversely proportional to T) and increases the volumetric flow rate $V = F_{\text{total}}/C_{\text{total}}$ (where $C_{\text{total}} = \rho_g/M_{\text{avg}}$), thereby increasing u and shortening the local residence time $\Delta\tau_j = \Delta z/u_j$.

Conversely, the mole reduction (1.5 to 1 mole per reaction) decreases total lbmols, which tends to decrease V and u , increasing $\Delta\tau_j$. In practice for SO₂ oxidation in adiabatic conditions, the temperature effect often dominates, leading to a net decrease in density, increase in u , and reduction in residence time, which can limit conversion in downstream slices and promote hotspots [1].

This variability enters the discretized equations through the convective terms: the upwind scheme incorporates u_j and u_{j-1} , so changes in density propagate inter-slice coupling. For example, in the mass balance ODE for slice j :

$$\varepsilon \frac{dC_{i,j}}{dt} = \frac{u_{j-1}C_{i,j-1} - u_j C_{i,j}}{\Delta z} + \nu_i \rho_b r_j$$

Here, density-induced variations in u affect the "inflow" and "outflow" rates, mimicking how reduced density (e.g., from heating) accelerates fluid through the slice, limiting reaction time and potentially creating hotspots or runaway conditions. Higher-order schemes or finer N are recommended when density changes are pronounced to avoid numerical dispersion errors [1].

Thus, the entire reactor is approximated as N CSTRs in series. This interpretation is particularly useful for understanding the numerical scheme: the method of lines with upwind differencing is mathematically equivalent to solving the dynamic mass, energy, and momentum balances for a cascade of well-mixed reactors. In the limit as $N \rightarrow \infty$ (and $\Delta z \rightarrow 0$), the cascade converges to the true plug-flow behavior. For finite N , the model introduces a small degree of axial backmixing (dispersion), which can be quantified via the Bodenstein number or Peclet number, but this is usually acceptable for large N and provides a robust, physically meaningful framework for simulation [1].

Discretization Details

For the convective terms (e.g., $\partial(uC_i)/\partial z$), an upwind finite difference scheme is recommended to ensure numerical stability in hyperbolic systems (where information propagates in the flow direction). For positive velocity $u > 0$, the first-order upwind approximation at slice j is:

$$\left(\frac{\partial(uC_i)}{\partial z} \right)_{z_j} \approx \frac{u_j C_{i,j} - u_{j-1} C_{i,j-1}}{\Delta z}$$

Higher-order schemes (e.g., second-order upwind or total variation diminishing schemes with flux limiters) may be used for improved accuracy, especially when sharp concentration or temperature fronts are expected.

The reaction terms ($\nu_i \rho_b r$) and heat release terms are evaluated locally at each slice using the current state values at z_j .

For the momentum balance, which is quasi-steady, the pressure drop is computed and integrated along z at each time step using the same discretization, treating it as an algebraic constraint coupled to the ODE system.

Applying this spatial discretization to all PDEs yields a system of approximately $3N$ ODEs (accounting for concentrations of reactive species, temperature, and pressure per slice; inert species may be lumped). The resulting system takes the standard form:

$$\frac{dy}{dt} = f(y, t)$$

where y is the global state vector containing all discretized variables ($C_{i,j}, T_j, P_j$ for all j). This ODE system is typically stiff due to fast reaction kinetics and strong coupling through variable velocity. The effective residence time per slice $\Delta\tau_j$ influences the stiffness: shorter residence times (higher velocity) require smaller time steps or more robust implicit solvers to maintain stability.

The choice of N (typically 50-200 slices) balances spatial resolution and computational cost. Convergence should be verified by successively refining Δz (increasing N) until axial profiles of conversion, temperature, and pressure stabilize. The residence time interpretation also aids in interpreting results: for example, a sudden increase in inlet temperature will produce a thermal wave that takes approximately $\tau(L)$ (total reactor residence time) to reach the outlet [1].

7.7 Numerical Solution Notes

The system is hyperbolic and suitable for the method of lines (MOL): discretize z (or catalyst weight $W = \rho_b A_c z$) into N slices, transforming the PDEs into approximately $3N$ ODEs in time. Employ a stiff solver (e.g., BDF or Radau) for integration. If gas holdup is minor (ε small), approximate accumulation terms involving $\varepsilon \partial/\partial t$ as zero to simplify, though this may underpredict transients [1].

Definition and Origin of y and f

The ODE system is written in the standard form:

$$\frac{dy}{dt} = f(y, t)$$

y is the global state vector (a column vector of length approximately $3N$), which contains the complete set of unknown variables at all spatial slices at any given time. Specifically:

$$y(t) = \begin{bmatrix} C_{\text{SO}_2,1} \\ C_{\text{O}_2,1} \\ C_{\text{SO}_3,1} \\ T_1 \\ P_1 \\ \vdots \\ C_{\text{SO}_2,N} \\ C_{\text{O}_2,N} \\ C_{\text{SO}_3,N} \\ T_N \\ P_N \end{bmatrix}$$

(For simplicity, the ordering may vary in code, but the vector collects all concentrations of reactive species, temperature, and pressure for every slice $j = 1$ to N . Inert species are typically lumped into the total flow calculation and do not require separate entries.)

The function $f(y, t)$ is the right-hand-side vector function that computes the time derivatives of all state variables. It is constructed directly from the discretized PDEs (Section 7.6). For each slice j , the components of f include:

- **Mass balance derivatives:** $\frac{dC_{i,j}}{dt} = \frac{u_{j-1}C_{i,j-1} - u_j C_{i,j}}{\varepsilon \Delta z} + \frac{\nu_i \rho_b r_j}{\varepsilon}$ (upwind convection plus reaction)
- **Energy balance derivative:** $\frac{dT_j}{dt} = \frac{-u \rho_g C_{p,g} (T_j - T_{j-1})}{\Delta z (\varepsilon \rho_g C_{p,g} + (1-\varepsilon) \rho_s C_{p,s})} + \frac{(-\Delta H_{Rx}) \rho_b r_j}{\varepsilon \rho_g C_{p,g} + (1-\varepsilon) \rho_s C_{p,s}}$ (discretized)
- **Pressure derivative** (from quasi-steady momentum): either set as algebraic constraint or included as a very fast ODE (large negative feedback term) to force rapid equilibration

Thus, $f(y, t)$ evaluates the right-hand sides of all discretized equations at the current state y and time t , using current values of u_j , r_j , ρ_g , etc., which depend on local $C_{i,j}$, T_j , and P_j .

The stiff solver (e.g., Radau) takes this system and integrates it forward in time from the current state $y(t_k)$ to $y(t_k + \Delta t)$, where Δt is adaptively chosen based on error control and stiffness. The Jacobian $J = \partial f / \partial y$ is the matrix of partial derivatives of each component of f with respect to each component of y . Because the equations are local (reaction terms depend only on slice j) and convective terms couple only neighboring slices (j to $j - 1$), J is sparse, with non-zero entries primarily along the main diagonal and the subdiagonal (for upwind convection). This sparsity is exploited for computational efficiency.

To solve this ODE system, employ a stiff solver such as Radau (a fully implicit Runge-Kutta method), which excels in handling the stiffness from fast kinetics or variable velocity. Radau uses the Jacobian matrix $J = \partial f / \partial y$, a sparse $3N \times 3N$ matrix capturing inter-slice couplings (e.g., convective terms link slice j to $j - 1$). The solver linearizes the system via Newton-Raphson iterations, solving $J \Delta y = -r$ (where r is the residual) at each stage, ensuring convergence even for large N (e.g., 100 slices yielding 300 ODEs). Update the Jacobian periodically (e.g., every 10 steps) to balance accuracy and computation [1].

Markers for determining reactor outlet conditions involve evaluating the state at the final slice N after each major time step Δt in the overall dynamic plant simulation. These outlet values (e.g., $C_{i,N}(t_k + \Delta t)$, $T_N(t_k + \Delta t)$, $P_N(t_k + \Delta t)$) serve as inputs to downstream units (e.g., absorbers or heat exchangers in the sulfuric acid flowsheet [1]), enabling coupled plant-wide dynamics. Use tolerances (e.g., relative change $< 10^{-4}$) as convergence markers to advance the global time step.

7.8 Using Runge-Kutta (RK) for Steady-State Initial Conditions

To establish initial conditions for the dynamic simulation, first compute the steady-state profiles by setting all time derivatives ($\partial / \partial t$) to zero in the PDEs, reducing them to a system of ordinary differential equations (ODEs) along z . These ODEs describe the spatial variations in concentrations, temperature, and pressure at

equilibrium (no transients). Integrate them using a fourth-order Runge-Kutta (RK4) method, which is effective for non-stiff or moderately stiff systems like this adiabatic case [1].

Procedure:

- Start at $z = 0$ with inlet values $(C_{i,0}, T_0, P_0)$
- Apply RK4 to march along z in small steps (Δz), evaluating the right-hand sides of the ODEs (e.g., reaction rate, heat release, pressure drop) at intermediate points per RK4 step for accuracy
- This yields profiles such as $C_i(z)$, $T(z)$, and $P(z)$ across the reactor length

Assignment to Slices:

Divide the reactor into N discrete slices along z . The steady-state value at the midpoint or end of each slice becomes the initial condition for that slice's variables at $t = 0$.

Benefits:

This ensures the dynamic simulation starts from a physically consistent steady state, avoiding unphysical transients. If the system is stiff (e.g., due to fast kinetics), combine RK with adaptive step sizing or switch to an implicit method for the steady-state solve.

Implementation Tip:

In code (e.g., Python with SciPy's `solve_ivp` using `RK45`), define the ODE function with the spatial derivatives, integrate from $z = 0$ to $z = L$ (reactor length), and interpolate results onto the slice grid [1].

7.9 Using Python Data Structures for Tracking Conditions

In Python implementations, employ data structures to systematically track initial and final conditions for each reactor slice across time intervals. Use NumPy arrays or Pandas DataFrames for efficiency: e.g., a 3D array

`states[time_step, slice_idx, var_idx]` where:

- `time_step` indexes discrete times t_k
- `slice_idx` ranges from 0 to $N - 1$
- `var_idx` covers variables (e.g., 0-3 for C_i , 4 for T , 5 for P)

At each Δt , store the initial slice states before integration and update with final states post-solver. For larger plant simulations, use dictionaries (e.g., `{'inlet': inlet_dict, 'outlet': outlet_dict}`) or HDF5 files for persistence, enabling checkpointing and analysis of transients like conversion waves [1].

For refinements, such as reincorporating cooling for specific operational scenarios or validating against plant data, consult specialized texts like Froment et al. (2010) or the referenced Fogler documentation [1]. Operational personnel should simulate startup sequences using this model to predict hotspot formation and adjust feed rates accordingly.

This represents the complete and updated version of Chapter 7 as of December 31, 2025. All equations are formatted in standard mathematical notation suitable for clean export to PDF. If you require the inclusion of the process flowsheet figure (from Image ID 7) or any additional sections, please advise.

References

[1] Fogler, H. S. (1992). *Elements of Chemical Reaction Engineering*, 2nd Edition. Prentice Hall, Englewood Cliffs, NJ. (Specific sections: R8.4 for industrial example of SO₂ oxidation, including rate laws, energy balances, and Polymath programs; Figures R8.4-1, R8.4-2, ER8.4-1.1; Equations R8.4-1, ER8.4-1, ER8.4-2, ER8.4-8.)